Sunbeam Institute of Information Technology

Preparatory Assignments – Logic Building Activity

Instructions

1. Solve the following assignments using **C, Java, or Python**, according to your convenience.
2. Focus on developing the **logic and problem-solving approach** rather than relying only on built-in functions.
3. Unless specifically mentioned, you may choose an appropriate approach and data structure.
4. Write clean, readable, and properly formatted code.
5. Test your programs with different input values, including boundary cases wherever applicable.

Submission Instructions

1. These assignments are intended for logic building and coding practice. Students are expected to solve the problems independently. The use of Google, AI tools, or other external assistance is not expected for solving these assignments.
2. All assignments must be completed and submitted in your GitHub repository during the first week of the course.
3. Detailed instructions regarding the GitHub repository, submission format, folder structure, and other requirements will be shared once the course begins.

Q1. Find the Maximum Number

Accept **n numbers through command-line arguments** and find and display the maximum number.

Note: Assume that at least one number is provided.

Q2. Factorial of a Number

Write a program to calculate the **factorial of a given non-negative integer**.

Q3. Fibonacci Series

Write a program to generate and display the **first n terms of the Fibonacci series**.

Q4. Student Grade

Write a program to calculate the grade of a student.

The student has **five subjects**, and marks for each subject are entered from the keyboard. Assume that each subject is evaluated **out of 20 marks**, making the total marks **out of 100**.

Assign the grade according to the following rules:

Total Marks	Grade
90–100	Ex
80–89	A
70–79	B
60–69	C
Below 60	F

Display the total marks and the corresponding grade.

Q5. Character Classification

Accept a string from the user and count/display the number of:

- Uppercase letters
- Lowercase letters
- Digits
- Other characters

Display an appropriate message for each category.

Q6. Matrix Multiplication

Write a program to perform **multiplication of two matrices**.

Accept the dimensions and elements of both matrices from the user. Check whether matrix multiplication is possible before performing the operation.

Condition: The number of columns in the first matrix must be equal to the number of rows in the second matrix.

Display the resulting matrix.

Q7. Multiplication Tables

Accept two numbers from the user and display the **multiplication tables for all numbers from the first number to the second number**.

Example:
For input `5` and `10`, display the multiplication tables of **5, 6, 7, 8, 9, and 10**.

Q8. Student Details Using a Structure/Class

Create a `Student` structure/class containing the following information:

- Student name
- Roll number
- Total marks

The roll number may contain both **letters and numbers**.

Accept the student details from the user and display the data as entered.

Q9. Number System Conversion

Accept an integer number from the user and display its:

- Binary equivalent
- Octal equivalent
- Hexadecimal equivalent

Sample:

```
Enter Number: 20

Given Number: 20
Binary equivalent: 10100
Octal equivalent: 24
Hexadecimal equivalent: 14
```

Q10. Sort Student Names

Read **at most 10 student names** and store them in an appropriate array/list.

Sort the names in **alphabetical order** and display the sorted names. Use appropriate library function for sorting.

Q11. Employee Details and Salary Increment

Create an `Employee` structure/class containing:

- First name
- Last name
- Monthly salary

Write appropriate functions/methods to:

1. Initialize employee details.
2. Display employee details.
3. Modify the employee's salary.
4. Calculate and display yearly salary.

Create **two Employee objects**.

Display the yearly salary of both employees. Then give each employee a **10% salary increase** and display their yearly salary again.

Q12. Reverse a String

Write a program to **reverse a given string** without using a built-in string-reversal function such as `strrev()` or slicing syntax.

Example:

```
Input:  SUNBEAM
Output: MAEBNUS
```

Q13. Find Duplicate Strings

Declare an array/list containing a few strings, including some duplicate strings.

Write a program to identify and display the **duplicate strings**.

If a string occurs more than once, display that string **only once** in the output.

Q14. String Palindrome

Write a program to check whether a given string is a **palindrome**.

A palindrome reads the same forward and backward.

Examples:

```
Input:  MADAM
Output: Palindrome

Input:  HELLO
Output: Not a Palindrome
```

Q15. Count Occurrences of Alphabets

Accept a string from the user and count the occurrence of **each alphabet**, without considering the difference between uppercase and lowercase letters.

Ignore spaces, digits, and special characters.

Display the count for each alphabet that occurs in the input.

Sample Input:

```
Welcome to SunBeam.
```

Sample Output:

```
A : 1
B : 1
C : 1
E : 3
L : 1
M : 2
N : 1
O : 2
S : 1
T : 1
U : 1
W : 1
```